

Programming Assignment 2: Basic programming in C++. Due: 11:59 pm @ 09/24/2025.

There are three components of this assignment. Each component asks you to write one C++ source file. I recommend you start with the provided skeleton code.

1 Isomorphic Strings

The purpose of this exercise is to determine whether two strings s and t are isomorphic.

Two strings s and t are isomorphic if the characters in s can be replaced to get t . All occurrences of a character must be replaced with another character while preserving the order of characters. No two characters may map to the same character, but a character may map to itself.

Example 1

```
Input: s = "egg", t = "add"
Output: true
```

Explanation: The strings s and t can be made identical by:

- Mapping 'e' to 'a'.
- Mapping 'g' to 'd'.

Example 2

```
Input: s = "foo", t = "bar"
Output: false
```

Explanation: The strings s and t cannot be made identical as 'o' would need to map to both 'a' and 'r'.

Example 3

```
Input: s = "paper", t = "title"
Output: true
```

2 Valid Parentheses

Your task is to implement a function that checks whether a given string contains valid nested parentheses. The string may include '(', ')', '[,]', and other characters.

Parentheses must be properly matched:

- Every opening parenthesis '(' must have a corresponding closing ')'.

- Every opening bracket '[' must have a corresponding closing ']'.

Parentheses must be correctly nested:

- "[()]" is invalid because '(' is closed before '['.
- "([])" is valid because they are properly nested.

Other characters in the string are ignored. If the string contains unmatched or incorrectly nested parentheses, return false. If the string contains no parentheses, consider it valid (true).

Examples

```
Input: "([])"
Output: true // Properly nested and matched
```

```
Input: "([)]"
Output: false // Incorrect nesting
```

```
Input: "(Hello [World])"
Output: true // Other characters ignored
```

3 Longest Consecutive Sequence

Given an unsorted array of integers `nums`, return the length of the longest consecutive elements sequence.

If you can, you should write an algorithm that runs in $O(n)$ time.

Example 1

```
Input: nums = [100, 4, 4, 200, 1, 3, 2]
Output: 4
```

Explanation: The longest consecutive elements sequence is [1, 2, 3, 4]. Therefore its length is 4.

Example 2

```
Input: nums = [0, 3, 7, 2, 5, 8, 4, 6, 0, 1]
Output: 9
```

Example 3

```
Input: nums = [1, 0, 1, 2]
Output: 3
```

Constraints

- $0 \leq \text{nums.length} \leq 10^5$
- $-10^9 \leq \text{nums}[i] \leq 10^9$

4 Instructions for Submission

Please read the following carefully.

1. Submit your programs in GradeScope. A link is provided from HuskyCT. Note that you can submit multiple files by drag and drop into the submission UI.
2. You must use the filenames as specified. If you change the filename, the Autograder won't work.
3. Please don't create your own files. Autograder won't know about the files you created (not in the provided starter code) and won't run test cases properly.
4. For automated grading, we mainly focus on functionalities. However, sometimes we will also test on (i) efficiency, (ii) code complexity, and (iii) code quality. We may use third-party tools for some of these tasks. While these tools have been around for some time, they can sometimes produce not-so-accurate reports about your code. If this is the case, contact the TAs about the issues.